LeetCode Practice

1 Hard: Two Pointers

Question, examples, then the six-phase interview approach with the brute force in phase 2

A split copy of the full All Questions book; every feature and link works the same

1
HARD

Patterns (in order)

1. Two Pointers (1)

How to use

- Tap any title in the Contents to jump to it. A topic name opens that topic's own divider page.
- Every page has < Prev | Contents | Next > at the top.
- Prev and Next move one question at a time; Contents returns here. Each topic starts on its own page.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

1 questions | tap a title to jump to it

Hard questions (1) Go >

Two Pointers (1) Go >

H 1. Find K-th Smallest Pair Distance **LC 719**

Hard questions (1)

HARD | TOPIC 1 OF 1

Two Pointers

1 Hard question start on the next page

Before this: this is the first topic
After this: this is the last topic

IN THIS TOPIC

1. Find K-th Smallest Pair Distance LC 719

HARD | Two Pointers | LeetCode #719 | 1 of 1 | 1/1

Find K-th Smallest Pair Distance

QUESTION

The distance of a pair of integers a and b is defined as the absolute difference between a and b.

Given an integer array nums and an integer k, return the kth smallest distance among all the pairs nums[i] and nums[j] where 0 <= i < j < nums.length.

Example 1

Input	`nums = [1,3,1], k = 1`
Output	`0`
Explanation	*Here are all the pairs:* *(1,3) -> 2* *(1,1) -> 0* *(3,1) -> 2* *Then the 1st smallest distance pair is (1,1), and its distance is 0.*

Example 2

Input	`nums = [1,1,1], k = 2`
Output	`0`

Example 3

Input	`nums = [1,6,1], k = 3`
Output	`5`

CONSTRAINTS

- n == nums.length
- 2 <= n <= 10^4
- 0 <= nums[i] <= 10^6
- 1 <= k <= n * (n - 1) / 2

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Given an integer array, find the kth smallest distance among all pairs (nums[i], nums[j]).

Distance = |nums[i] - nums[j]|. Right?"

"A few quick questions:

Are pair indices distinct (i != j)? Yes. Return the kth smallest, not the pair itself?"

"Let me walk: nums=[1,3,1], k=1 -> all dists: [0,2,2]. 1st smallest = 0 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
distances = |a - b| for every pair
sort the distances
return the k-th
```

TIME	SPACE
O(n^2 log n)	O(n^2)

WHY IT IS SLOW

Builds and sorts about n^2/2 values. Sort the array, binary search the distance d, and count pairs with distance <= d with two pointers: O(n log n + n log(max - min)).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is generating all O(n²) distances.

Binary search on the answer (distance d):

Count pairs with distance <= d using sorted array + two pointers or sliding window.

Find smallest d where count >= k.

Time: O(n log n + n log W) where W = max_dist. Space: O(1)."

Key insight: binary search on the ANSWER distance d, with a sliding-window count of pairs having distance <= d, makes each feasibility check O(n) — total O(n log W) vs O(n² log n) brute force.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _719_FindKthSmallestPairDistance:
    def smallestDistancePair(self, nums, k):
        nums.sort()
        n = len(nums)
        def count_pairs_le(mid):
            count = left = 0
            for right in range(n):
                while nums[right] - nums[left] > mid:
                    left += 1
                count += right - left
            return count
        lo, hi = 0, nums[-1] - nums[0]
        while lo < hi:
            mid = (lo + hi) // 2
            if count_pairs_le(mid) >= k:
                hi = mid
            else:
                lo = mid + 1
        return lo
```

PHASE 5 TEST & DEBUG

"Let me trace how the left/right pointers move and why the invariant stays true."

nums=[1,3,1], k=1 -> sorted=[1,1,3]. lo=0,hi=2.

mid=1: count_pairs_le(1): right=0->0.
right=1(1):1-1=0<=1->count=1.
right=2(3):3-1=2>1->left=1.count+=1=2.

Wait: right=2, left slides while nums[2]-nums[left]>1.
nums[2]-nums[1]=2>1->left=2. count+=0.

count=1. 1>=1->hi=1. mid=0: count=0<1->lo=1. lo=hi=1? No:
lo=0,hi=1,mid=0->count=count_pairs_le(0).

1-1=0<=0->count=1. 1>=1->hi=0. lo=hi=0. return 0 ok

Two-pointer sliding window counts pairs with distance <= mid in O(n)

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n log n + n log W)	**O(1)**

Binary search on the answer avoids materializing all distances.

Follow-up: Kth Smallest Sum (#373) — sorted pairs from two arrays; priority queue.

Follow-up: Kth Smallest in Multiplication Table (#668) — same binary search pattern.